

Lecture 2 - Dart OOP I

Classes, Objects, Inheritance, Polymorphism

Lab: Matrix, Vector Hierarchy

Fettah KIRAN

Department of AI and Data Engineering

Yıldız Technical University

fkiran@yildiz.edu.tr

Overview

1. Recap Quiz from Week 1
2. Dart Setup: DartPad, SDK, VS Code
3. Variables & Null Safety Essentials
4. Classes, Objects & Constructors
5. Fields, Methods, Getters/Setters, Static
6. Inheritance: extends, super, @override
7. Polymorphism, Abstract Preview, Implements
8. Lab: Vector2D → Vector3D, Matrix → SparseMatrix

Why Dart for Flutter?

- **One language** for UI + logic: no JS bridge, compiles to ARM/x64 native
- **OOP + modern**: classes, mixins, generics, null safety, async built-in
- **Hot Reload friendly**: JIT in dev, AOT in release — same code
- **Math friendly**: operator overloading (`+`, `*`, `[]`) — perfect for Vector/Matrix lab

Dart Setup (Zero Excuses)

- **Option A — DartPad:** <https://dartpad.dev> — runs in browser, zero install, all lab code runs here
- **Option B — Local:** `dart --version` (ships with Flutter SDK) → Dart 3.x
- **VS Code:** Flutter + Dart extensions → `.dart` files get run/debug/lint free
- Check now: `flutter doctor` clean? `dart --version` prints?

Ex 1 — Variables & Null Safety (Run in DartPad)

```
void main() {  
  var name = 'MTM4681';           // inferred String, non-nullable  
  int? score;                     // nullable: currently null  
  late String team;               // promised to init later  
  team = 'VectorSquad';  
  print('$name | $team | ${score ?? 0}');  
  // score! -> crash if null! Use ?? or if-check instead.  
  const pi = 3.14159;             // compile-time const  
  final now = DateTime.now();     // run-time final  
}
```

- `?` = may be null, `!` = trust me non-null (dangerous), `late` = init later
- `required` named params — next slides in constructors

Null Safety in 60 Seconds

- **Non-nullable by default:** `int x;` must be assigned before use
- `int? y` can hold null → check with `y == null`, `y ?? 0`, `y?.abs()`
- `late` delays init — useful for fields set in `initState()` later
- Rule of thumb: never use `!` unless you just null-checked on the line above

Ex 2 — First Class: Vector2D

```
class Vector2D {  
  double x, y; // fields  
  Vector2D(this.x, this.y); // generative constructor  
  double get norm => normOf(x, y); // getter  
  static double normOf(double a, double b) =>  
    (a * a + b * b); // placeholder, sqrt below  
  void translate(double dx, double dy) {  
    x += dx; y += dy; // method mutates  
  }  
}  
  
void main() {  
  var v = Vector2D(3, 4);  
  v.translate(1, 1);  
  print('${v.x}, ${v.y}'); // 4.0, 5.0  
}
```

- `this.x` shorthand assigns params to fields
- Try in DartPad: add `import 'dart:math';` and use `sqrt()`

Constructors: Generative + Named

```
import 'dart:math';
class Vector2D {
  double x, y;
  Vector2D(this.x, this.y);
  Vector2D.zero() : x = 0, y = 0;           // named constructor
  Vector2D.polar(double r, double angle)
    : x = r * cos(angle), y = r * sin(angle); // initializer list
  double get norm => sqrt(x * x + y * y);
}
void main() {
  print(Vector2D.zero().norm);           // 0.0
  print(Vector2D.polar(5, 0).x);         // 5.0
}
```

- Named constructors clarify intent: `.zero()`, `.polar()`, `.fromJson()`
- Initializer list (`:`) runs before body — required for `final` fields

Initializer List + const

```
class Vector2D {  
    final double x, y;           // immutable!  
    const Vector2D(this.x, this.y); // const constructor  
    const Vector2D.zero() : x = 0, y = 0;  
    double get norm => x * x + y * y;  
}  
void main() {  
    const a = Vector2D(3, 4);  
    const b = Vector2D(3, 4);  
    print(identical(a, b));      // true: canonicalized  
}
```

- `final` = set once; `const` = compile-time constant, canonicalized
- Use immutable vectors/matrices — safer for Flutter state

Factory Constructors

```
class Vector2D {  
    final double x, y;  
    const Vector2D(this.x, this.y);  
    factory Vector2D.fromJson(Map<String, dynamic> j) {  
        return Vector2D((j['x'] as num).toDouble(),  
                        (j['y'] as num).toDouble());  
    }  
    factory Vector2D.zero() => const Vector2D(0, 0); // may cache  
}  
void main() {  
    final v = Vector2D.fromJson({'x': 3, 'y': 4});  
    print('${v.x}, ${v.y}');  
}
```

- `factory` can return cached instance, subclass, or parse logic
- Week 6 preview: `fromJson` is how REST API math data becomes objects

Ex 3 — Getters, Setters, Methods

```
import 'dart:math';
class Vector2D {
  double x, y;
  Vector2D(this.x, this.y);
  double get norm => sqrt(x * x + y * y);
  set norm(double v) { // scale to length v
    final n = norm;
    if (n == 0) return;
    x = x / n * v; y = y / n * v;
  }
  double dot(Vector2D o) => x * o.x + y * o.y;
}
void main() {
  var v = Vector2D(3, 4); // norm 5
  v.norm = 10; // setter rescales
  print(v.norm.toStringAsFixed(1)); // 10.0
}
```

- Getter computes, setter validates/rescales — encapsulation without boilerplate

Static Members

- `static` belongs to the **class**, not the instance — `Vector2D.zero()`, `Matrix.identity(3)`
- Use for: constants (`static const double eps = 1e-9`), helpers (`normOf`), factories/caches
- Cannot use `this` inside static — no instance yet
- Lab tip: `static Vector2D parse(String s)` for `"3,4"` input

Ex 4 — Inheritance: Vector3D extends Vector2D

```
import 'dart:math';
class Vector2D {
  double x, y;
  Vector2D(this.x, this.y);
  double get norm => sqrt(x * x + y * y);
}
class Vector3D extends Vector2D {
  double z;
  Vector3D(super.x, super.y, this.z);    // super-parameters (Dart 2.17+)
  @override
  double get norm => sqrt(x * x + y * y + z * z);
}
void main() {
  Vector2D v = Vector3D(1, 2, 2);        // upcast: IS-A
  print(v.norm);                        // 3.0 -> dispatches to Vector3D!
}
```

- `extends` = IS-A; `super.x` forwards to parent constructor
- `@override` catches signature typos at analyze time

super & @override Rules

- `super(...)` or `super.x` must init parent before child body runs
- `@override` required intent: `toString()`, `==`, getters like `norm`
- Parent private (`_field`) is library-private — child in same file sees it, other files don't
- Liskov: anywhere `Vector2D` expected, `Vector3D` must work — don't break `norm` contract

Ex 5 — Polymorphism via Subtype

```
import 'dart:math';
abstract class Shape { double area(); }
class Circle extends Shape {
  final double r; Circle(this.r);
  @override double area() => pi * r * r;
}
class Rect extends Shape {
  final double w, h; Rect(this.w, this.h);
  @override double area() => w * h;
}
void main() {
  final List<Shape> shapes = [Circle(1), Rect(2, 3)];
  for (final s in shapes) print(s.area()); // 3.14..., 6.0
}
```

- Same call `area()`, different behavior — **polymorphism**
- Math twin: `List<Vector2D> = [Vector2D, Vector3D]` then `.norm` in a loop

Abstract Class Preview

- `abstract class Shape { double area(); }` — no instances, subclasses must implement
- Week 3 deep-dive: `abstract class Solver { Vector2D solve(...); }` + Newton vs Bisection
- Today: use abstract to force `Vector` subclasses to define `norm`
- Abstract $\neq$ interface: abstract can hold state + concrete helpers

Interface via implements

- Dart has no `interface` keyword — every class is an interface
- `class Circle implements Shape` must re-implement **everything** (no inherited code)
- `extends` = reuse code; `implements` = guarantee API; `with` (Week 3) = mixin reuse
- Lab: `class SparseMatrix implements Matrix` ? Prefer `extends` unless hiding storage

Ex 6 — Cascades (..)

```
class Matrix {  
  final List<List<double>> a;  
  Matrix(this.a);  
  void scale(double k) {  
    for (var r = 0; r < a.length; r++) {  
      for (var c = 0; c < a[r].length; c++) a[r][c] *= k;  
    }  
  }  
  void printM() => print(a);  
}  
void main() {  
  final m = Matrix([[1, 2], [3, 4]])  
    ..scale(10)  
    ..printM();  
}                                     // [[10,20],[30,40]]
```

- `..` chains calls on the **same object** — replaces verbose temp vars
- Flutter twin: `Paint()..color = red..strokeWidth = 2` (Week 10)

Enums (Enhanced)

- `enum Op { add, mul, transpose }` — finite choices for calculator/matrix ops
- Enhanced enums hold fields/methods: `enum Norm { l1('Manhattan'), l2('Euclid'); ... }`
- Use in `switch` — analyzer warns if you miss a case
- Lab: `enum MatrixKind { dense, sparse, identity }`

Null Safety in OOP

```
class Student {  
  final String name;  
  String? email;           // optional -> nullable  
  late final String id;    // set in register()  
  Student({required this.name, this.email});  
  void register(String newId) => id = newId;  
}  
void main() {  
  var s = Student(name: 'Ada');           // email null OK  
  s.register('YTU-001');  
  print(s.email ?? 'no email');          // ?? fallback  
}
```

- `required` named params kill null bugs at call site
- `late final` for DI / `initState` patterns in Flutter

Ex 7 — toString / == / hashCode

```
class Vector2D {  
    final double x, y;  
    const Vector2D(this.x, this.y);  
    @override String toString() => 'Vector2D($x, $y)';  
    @override bool operator ==(Object o) =>  
        o is Vector2D && o.x == x && o.y == y;  
    @override int get hashCode => Object.hash(x, y);  
}  
void main() {  
    print(const Vector2D(3, 4));           // Vector2D(3.0, 4.0)  
    print(const Vector2D(3, 4) == const Vector2D(3, 4)); // true  
}
```

- Always override all three together — Set / Map / == in tests depend on it
- Lab acceptance: `Matrix([...]) == Matrix([...])` must compare values, not identity

Packages & Imports

- `import 'dart:math';` → `sqrt`, `pi`, `Random` — core SDK
- `import 'package:vector_lab/vector.dart';` → your own files (Week 3 multi-file lab)
- Relative: `import 'vector.dart';` inside same folder/lib
- `as`, `show`, `hide`: `import 'dart:math' as m show sqrt;` avoids name clashes

dart format & analyze

- `dart format .` — canonical style, required before Final submission
- `dart analyze` — catches missing `@override`, dead null-checks, unused imports
- VS Code: format-on-save ON; problems panel = analyzer live
- CI habit: format + analyze clean = professionalism points on project rubric

Math Focus 1 — Vector2D Complete

```
import 'dart:math';
class Vector2D {
  final double x, y;
  const Vector2D(this.x, this.y);
  const Vector2D.zero() : x = 0, y = 0;
  double get norm => sqrt(x * x + y * y);
  double dot(Vector2D o) => x * o.x + y * o.y;
  Vector2D operator +(Vector2D o) => Vector2D(x + o.x, y + o.y);
  Vector2D operator *(double k) => Vector2D(x * k, y * k);
  @override String toString() => '($x, $y)';
}
void main() {
  const v = Vector2D(3, 4);
  print((v + const Vector2D(1, 1)) * 2); // (8.0, 10.0)
}
```

Math Focus 2 — Matrix + Operators

```
class Matrix {  
    final List<List<double>> a;  
    Matrix(this.a);  
    int get rows => a.length;  
    int get cols => a[0].length;  
    Matrix operator +(Matrix o) {  
        return Matrix(List.generate(rows,  
            (r) => List.generate(cols, (c) => a[r][c] + o.a[r][c])));  
    }  
    double operator [] (List<int> idx) => a[idx[0]][idx[1]];  
}  
void main() {  
    final m = Matrix([[1, 2], [3, 4]]);  
    print((m + m)[[1, 1]]);           // 8.0  
}
```

- operator `+`, `*`, `[]` make math read like math
- Lab: add operator `*` (`matxmat + matxvec`) with dimension assert

Inheritance in Math: Vector3D

```
import 'dart:math';
class Vector3D extends Vector2D {
  final double z;
  const Vector3D(super.x, super.y, this.z);
  @override double get norm => sqrt(x * x + y * y + z * z);
  double dot3(Vector3D o) => x * o.x + y * o.y + z * o.z;
  @override String toString() => '($x, $y, $z)';
}
void main() {
  const v = Vector3D(1, 2, 2);
  print(v.norm); // 3.0
}
```

- Needs `Vector2D` non-const or const-compatible chain — keep fields `final`
- Liskov check: every `Vector2D` API still sensible on `Vector3D` ?

Inheritance in Math: SparseMatrix

```
class SparseMatrix extends Matrix {  
  SparseMatrix(super.a);           // mostly zeros  
  factory SparseMatrix.identity(int n) {  
    final a = List.generate(n,  
      (r) => List.generate(n, (c) => r == c ? 1.0 : 0.0));  
    return SparseMatrix(a);  
  }  
  int get nonZeros =>  
    a.expand((r) => r).where((v) => v != 0).length;  
  @override String toString() => 'Sparse($rows×$cols, nz=$nonZeros)';  
}  
void main() {  
  print(SparseMatrix.identity(3)); // Sparse(3×3, nz=3)  
}
```

- Subclass adds storage-aware API (`nonZeros`) without rewriting `+` / `*`
- Extension: override `*` with sparse-fast path (skip zeros)

Ex 8 — Polymorphism Demo: Norms

```
import 'dart:math';
void main() {
  final List<Vector2D> vs = [
    const Vector2D(3, 4),           // norm 5
    const Vector3D(1, 2, 2),       // norm 3
    Vector2D.polar(5, 0),          // norm 5
  ];
  for (final v in vs) print('${v.runtimeType}: ${v.norm}');
  final total = vs.fold(0.0, (s, v) => s + v.norm);
  print('total norm = $total');    // 13.0
}
```

- One `List<Vector2D>`, three runtime types, one `.norm` call
- This is the exact pattern behind Week 3 `List<Solver>` and Flutter `List<Widget>`

Common Errors (Save 30 min)

- `const` needs all-final fields + `const super` — else "Can't define const constructor"
- `!` on nullable after async gap — value may turn null; use local copy + null check
- Missing `@override` + signature drift → silent overload, wrong dispatch
- `==` without `hashCode` → `Set` dedup fails in tests
- Mutable `List<List<double>>` aliasing — copy in constructor (`List.of`) for Matrix

Quick Check

- **T1:** `const Vector2D(3,4) == const Vector2D(3,4)` — true or false? Why without custom `==`?
 - **T2:** Where must the initializer list go — before or after the constructor body? What can it init that body cannot?
 - **T3:** `Vector2D v = Vector3D(1,2,2); v.norm` — which `norm` runs? What OOP term is this?
-
- Answers: (1) true only with canonical const, else identity; custom `==` needed for non-const (2) before body (`:`), inits `final` (3) `Vector3D.norm` — polymorphism/dynamic dispatch

Dart vs Java / Python (For Seniors)

	Dart	Java	Python
Null safety	Sound (<code>?</code> , <code>!</code> , <code>required</code>)	Optional/ <code>@Nullable</code>	None (None everywhere)
Constructors	Generative/named/factory/const	Overloaded only	<code>__init__</code> only
Inheritance	Single + <code>implements</code> / <code>with</code>	Single + interfaces	Multiple
Operators	Overloadable (<code>+</code> , <code>[]</code>)	Not (except String+)	Overloadable (<code>__add__</code>)
Best for course	Native Flutter compile	Verbose for mobile	Slow on phones

- You know OOP — focus on Dart idioms: `..`, `?.`, `??`, `const`, `factory.fromJson`

Mixin Teaser → Week 3

- `mixin Printable { String dump(); }` then `class Matrix with Printable`
- Mixins = reuse without inheritance tree — Week 3: `Validatable`, `Serializable`
- Today: master `extends` + `implements`; Week 3 adds `with` + `Future<Solution>`

Predict the Output

```
class A { String hi() => 'A'; }
class B extends A { @override String hi() => 'B'; }
void main() {
  A x = B();
  print(x.hi());
  final m = Matrix([[1, 2], [3, 4]]);
  print((m + m)[[0, 1]]);
}
```

- What prints? (Answer: B then 4.0) — why is the static type A but behavior B?

Lab — Matrix & Vector Hierarchy

60 min: build, overload, extend, demo

Lab Step 1 — Vector2D (15 min)

1. DartPad → new Pad, `import 'dart:math';`
2. `final x, y;` `const` constructor + `.zero()` + `.polar(r, angle)`
3. `norm` getter, `dot()`, `+`, `*` (scalar), `toString`, `==` + `hashCode`
4. Test: `Vector2D(3,4).norm == 5`, `(v + zero) * 2`

- **Done when:** 4 asserts pass in `main()`. Show neighbor.

Lab Step 2 — Matrix + Operators (20 min)

1. `Matrix(List<List<double>> a)` with `rows` / `cols` getters
2. `operator +` (same dims, else `assert` / `throw ArgumentError`)
3. `operator *`: scalar `Matrix * double` AND `matxmat` (triple loop, `assert cols == o.rows`)
4. `operator []([r, c])` read + `toString()` grid print
5. Test: `identity(2) * m == m`, `(m + m)[[1,1]] == 8.0`

Lab Step 3 — Inheritance (15 min)

1. `Vector3D` extends `Vector2D` + `z`, override `norm`, add `dot3` / `cross`
2. `SparseMatrix` extends `Matrix` + `identity(n)` factory + `nonZeros` getter
3. Polymorphism: `List<Vector2D> vs = [Vector2D(3,4), Vector3D(1,2,2)]` → loop `.norm`, sum it
4. Paste runtime types: `v.runtimeType` proves dispatch

Lab Acceptance Criteria

- [] `Vector2D(3,4).norm == 5.0`, `Vector3D(1,2,2).norm == 3.0`
- [] `Matrix +`, `*` (scalar + matxmat), [] all work with asserts on bad dims
- [] `==` / `hashCode` / `toString` overridden for Vector + Matrix
- [] `dart analyze` zero warnings, `dart format` applied
- [] Polymorphism demo: mixed list loop prints correct norms

Show TA/instructor before leaving — counts toward participation.

Extension Challenge

- Transpose getter, `determinant2x2`, `Matrix.identity(n)` static
- `SparseMatrix *` fast path skipping zeros — benchmark 100×100
- `Vector2D.fromJson` / `toJson` — prepares Week 6 HTTP + JSON
- Normalize setter: `v.norm = 1.0` (unit vector) — handle zero-vector guard
- First team to demo sparse speedup picks presentation slot Week 13!

Homework

- Finish lab file → rename `vector_matrix.dart`, keep for Week 3 (`Solver` builds on it)
- Run `flutter doctor -v` — must be green (except optional desktop/Xcode) by Week 3
- Read: Dart tour (<https://dart.dev/language>) — classes + enums sections
- Form project team (3-5) — list + 2 app ideas due Week 3

Practice Resources

- **DartPad** — all 8 lab examples run here tonight
- **Dart Language Tour** — classes, constructors, enums, null safety
- **Dart Cheatsheet + Operator overloading** — `+`, `[]`, `==` rules
- **Flutter Codelab** — re-run Counter, add a `Vector` print

Debugging Tips

- Red squiggle? Hover → read analyzer message → `dart analyze` in terminal
- `print(v.runtimeType)` reveals which subclass actually runs
- Breakpoint in VS Code on `norm` getter — step into override to see dispatch
- Aliasing bug? `Matrix(List.of(a))` deep-copy rows before mutating

Readiness & Teams — Due Week 3!

- **Teams of 3-5** (2 or 6 only with written approval) — list due Week 3
- **Readiness Survey** (same form as Week 1 — update if team/idea changed)
- https://docs.google.com/forms/d/e/1FAIpQLSeZktvCwdnPSH7dtbq_MKarB2qLdd7uT1FUzBIU4d-jBZDL-A/viewform?usp=publish-editor
- Need members? Stay 5 min after class — matchmaking corner

Next Week Preview

- **Week 3: Dart OOP II — Abstraction, Encapsulation, Mixins, Async**
- `abstract class Solver`, generics, `Future` — Lab: numerical method class (Bisection/Newton)
- Bring: working `vector_matrix.dart` + `flutter doctor` clean
- Pre-read: `Future` vs `Stream` (one page, Dart tour async)

Questions & Discussion

Thank you!

Install check: flutter doctor + practice on given HW — see you Thursday 9:00!